



Guía n.º 1.1.4

Actividad: Paradigma Orientado a Objetos

Abstracción de clases, atributos y métodos a partir de casos

Guía n.º	1.1.4
Nombre de la guía	Actividad de modelado básico de clases en POO
Sigla	DSY1102
Asignatura	Desarrollo Orientado a Objetos
Experiencia de Aprendizaje	Analizar situaciones cotidianas para abstraer clases con atributos y métodos.
Tiempo	90 minutos
Modalidad de Trabajo	Individual o en parejas
Indicadores de Logro	Identifica objetos en un caso, abstrae una clase pertinente, selecciona atributos y métodos relevantes y justifica sus decisiones de modelado.

Propósito de la guía

Desarrollar la habilidad de abstraer una clase a partir de objetos del mundo real, identificando solo la información y las acciones más importantes para resolver un problema en Java.

La guía contiene 10 casos y 13 ejercicios. En cada ejercicio el estudiante deberá proponer una clase, sus atributos, sus métodos y un constructor básico.

Instrucciones generales

- Lee con atención cada caso antes de responder.
- En cada ejercicio identifica una clase principal.
- Escribe al menos 4 atributos y 3 métodos por ejercicio, salvo que el caso indique otra cantidad.
- Propón un constructor coherente con la clase definida.
- Usa nombres claros y relacionados con el contexto del caso.
- No incluyas detalles irrelevantes del mundo real; selecciona solo lo necesario para el problema.

Formato sugerido de respuesta para cada ejercicio

1. Nombre de la clase
2. Descripción breve de la responsabilidad de la clase
3. Lista de atributos
4. Lista de métodos
5. Constructor propuesto
6. Justificación breve: ¿qué datos y acciones fueron seleccionados y por qué?



Caso 1: Biblioteca escolar

Ejercicio 1: Clase Libro

Situación: La biblioteca del establecimiento necesita registrar los libros que presta a los estudiantes. De cada libro interesa saber datos útiles para identificarlo y saber si está disponible.

Consigna: A partir del objeto real libro, abstraer una clase llamada Libro o un nombre equivalente. Define atributos y métodos que permitan representar y usar ese libro dentro de un sistema simple de biblioteca.

- Propón el nombre de la clase.
- Escribe al menos 4 atributos.
- Escribe al menos 3 métodos.
- Propón un constructor básico.
- Explica en 2 o 3 líneas qué detalles del objeto real dejaste fuera y por qué.

Clase propuesta	Libro
Atributos	name, author, stock, SUK_code
Métodos	getSuk_code canRetire setStock
Constructor	name, author, stock, SUK_code,
Justificación	Deje de lado el código de unidad, no se me ocurrió una forma de hacer que un libro tenga diferentes copias, se me ocurre que eso lo podría generar la BD con una columna capaz de recibir array o json. La verdad no se bien como lo haría una biblioteca real y me resulta interesante pensar como lo haría.



Ejercicio 2: Clase Préstamo

Situación: Además de registrar libros, la biblioteca necesita guardar cada préstamo realizado a un estudiante, indicando fechas y estado de devolución.

Consigna: Abstrae una clase Préstamo. Piensa qué información debe guardar el préstamo y qué acciones básicas debería permitir dentro del sistema.

- Propón el nombre de la clase.
- Escribe al menos 4 atributos.
- Escribe al menos 3 métodos.
- Propón un constructor básico.
- Explica en 2 o 3 líneas qué detalles del objeto real dejaste fuera y por qué.

Clase propuesta	Prestamo
Atributos	student , libro , start_date, end_date, status
Métodos	getStudent, getBook, setEnd_date, setStatus
Constructor	student , libro , start_date, end_date
Justificación	Deje fuera el id_prestamo. Me parece que es un detalle importante, pero que para el contexto del ejercicio y nivel de clase, aun no hace falta.



Caso 2: Tienda en línea

Ejercicio 3: Clase Producto

Situación: Una tienda virtual vende productos tecnológicos. Cada producto debe poder mostrarse en pantalla y actualizar su stock cuando ocurre una venta.

Consigna: Abstrae una clase Producto. Selecciona los datos esenciales del producto y las acciones mínimas que debería tener.

- Propón el nombre de la clase.
- Escribe al menos 4 atributos.
- Escribe al menos 3 métodos.
- Propón un constructor básico.
- Explica en 2 o 3 líneas qué detalles del objeto real dejaste fuera y por qué.

Clase propuesta	product
Atributos	name, code, stock, last_update
Métodos	venta, addStock, getName, getStock
Constructor	name, code, stock, last_update
Justificación	Deje fuera la ubicacion. Igual que en casos anteriores, no se me ocurre bien resolver los casos 1 es a muchos



Caso 3: Banco

Ejercicio 4: Clase CuentaBancaria

Situación: Un banco necesita representar cuentas de ahorro para poder consultar saldo, depositar dinero y girar fondos.

Consigna: Abstrae una clase CuentaBancaria. Define atributos relacionados con la cuenta y métodos que permitan operarla de forma básica.

- Propón el nombre de la clase.
- Escribe al menos 4 atributos.
- Escribe al menos 3 métodos.
- Propón un constructor básico.
- Explica en 2 o 3 líneas qué detalles del objeto real dejaste fuera y por qué.

Clase propuesta	CuentaBancaria
Atributos	full_name, rut, balance, email
Métodos	retirar, ingresar, mostrarSaldo
Constructor	full_name, rut, email
Justificación	Deje fuera datos como la fecha de creacion, o las tarjetas asociadas. Creo que complicarian un codigo del nivel actual



Caso 4: Gestión académica

Ejercicio 5: Clase Estudiante

Situación: En una aplicación académica se desea registrar a cada estudiante con sus datos principales y algunas operaciones básicas del sistema.

Consigna: Abstrae una clase Estudiante. Incluye atributos relacionados con identificación y rendimiento, y métodos coherentes con el contexto.

- Propón el nombre de la clase.
- Escribe al menos 4 atributos.
- Escribe al menos 3 métodos.
- Propón un constructor básico.
- Explica en 2 o 3 líneas qué detalles del objeto real dejaste fuera y por qué.

Clase propuesta	Estudiante
Atributos	first_name, notas de: matematicas historia lenguaje ingles, promedio, email
Métodos	setNotas, calcularPromedio, get promedio
Constructor	first_name, notas de: matematicas historia lenguaje ingles, email
Justificación	Deje fuera la creacion de listas de notas por cada curso. Traerian varios problemas para el nivel actual y no sabria manejarlas aun.



Ejercicio 6: Clase Curso

Situación: El mismo sistema necesita representar cursos para saber su nombre, cupos y estudiantes inscritos.

Consigna: Abstrae una clase Curso. Piensa qué características debería tener el curso y qué acciones básicas podría realizar.

- Propón el nombre de la clase.
- Escribe al menos 4 atributos.
- Escribe al menos 3 métodos.
- Propón un constructor básico.
- Explica en 2 o 3 líneas qué detalles del objeto real dejaste fuera y por qué.

Clase propuesta	Curso
Atributos	name, teacher, estudiantes, cupos
Métodos	agregarEstudiante, eliminar estudiante, cuposDisponibles
Constructor	name, teacher, cupos
Justificación	Deje fuera ayudantes, fechas de evaluaciones o contenido, clases (horarios)



Caso 5: Veterinaria

Ejercicio 7: Clase Mascota

Situación: Una veterinaria registra mascotas para atender controles y vacunas. Necesita almacenar información útil para identificar a cada animal y su estado general.

Consigna: Abstrae una clase Mascota. Selecciona atributos y métodos que sean relevantes para una ficha simple de atención.

- Propón el nombre de la clase.
- Escribe al menos 4 atributos.
- Escribe al menos 3 métodos.
- Propón un constructor básico.
- Explica en 2 o 3 líneas qué detalles del objeto real dejaste fuera y por qué.

Clase propuesta	Mascota
Atributos	name, type, owner, age, lastVacum, nextControl
Métodos	getName, getTipe, setNextControl
Constructor	name, type, owner, age
Justificación	Deje fuera el calendario de vacunacion. Creo que un siguiente paso seria bueno crear al clase vacuna, y en el objeto mascota, tener una lista de vacunas.



Caso 6: Delivery de comida

Ejercicio 8: Clase Pedido

Situación: Una aplicación de delivery debe registrar pedidos realizados por clientes, calcular un total y actualizar su estado durante el proceso de entrega.

Consigna: Abstrae una clase Pedido. Define los datos más importantes y las acciones necesarias para controlar el pedido.

- Propón el nombre de la clase.
- Escribe al menos 4 atributos.
- Escribe al menos 3 métodos.
- Propón un constructor básico.
- Explica en 2 o 3 líneas qué detalles del objeto real dejaste fuera y por qué.

Clase propuesta	
Atributos	
Métodos	
Constructor	
Justificación	



Caso 7: Cine

Ejercicio 9: Clase EntradaCine

Situación: Un cine vende entradas para distintas funciones. La entrada debe contener información útil para validar acceso y conocer su valor.

Consigna: Abstrae una clase EntradaCine. Considera atributos y métodos que permitan representar una entrada y operarla de forma simple.

- Propón el nombre de la clase.
- Escribe al menos 4 atributos.
- Escribe al menos 3 métodos.
- Propón un constructor básico.
- Explica en 2 o 3 líneas qué detalles del objeto real dejaste fuera y por qué.

Clase propuesta	
Atributos	
Métodos	
Constructor	
Justificación	



Caso 8: Clínica

Ejercicio 10: Clase Paciente

Situación: Una clínica requiere un registro básico de pacientes para identificarlos, saber su edad y consultar información relevante para la atención.

Consigna: Abstrae una clase Paciente. Elige los atributos más útiles y propone métodos acordes al contexto.

- Propón el nombre de la clase.
- Escribe al menos 4 atributos.
- Escribe al menos 3 métodos.
- Propón un constructor básico.
- Explica en 2 o 3 líneas qué detalles del objeto real dejaste fuera y por qué.

Clase propuesta	
Atributos	
Métodos	
Constructor	
Justificación	



Ejercicio 11: Clase Cita Médica

Situación: La clínica también necesita registrar citas médicas con fecha, hora y estado de atención.

Consigna: Abstrae una clase CitaMedica. Piensa qué información debe tener una cita y qué acciones básicas debería soportar.

- Propón el nombre de la clase.
- Escribe al menos 4 atributos.
- Escribe al menos 3 métodos.
- Propón un constructor básico.
- Explica en 2 o 3 líneas qué detalles del objeto real dejaste fuera y por qué.

Clase propuesta	
Atributos	
Métodos	
Constructor	
Justificación	



Caso 9: Transporte escolar

Ejercicio 12: Clase BusEscolar

Situación: Un colegio desea controlar los buses escolares, considerando capacidad, recorrido y estado del vehículo.

Consigna: Abstrae una clase BusEscolar. Define atributos y métodos que permitan administrar de forma simple un bus dentro del sistema.

- Propón el nombre de la clase.
- Escribe al menos 4 atributos.
- Escribe al menos 3 métodos.
- Propón un constructor básico.
- Explica en 2 o 3 líneas qué detalles del objeto real dejaste fuera y por qué.

Clase propuesta	
Atributos	
Métodos	
Constructor	
Justificación	



Caso 10: Gimnasio

Ejercicio 13: Clase Socio

Situación: Un gimnasio registra socios para controlar su plan, vigencia de pago y asistencia a entrenamientos.

Consigna: Abstrae una clase Socio. Selecciona la información principal y las acciones más importantes para esta clase.

- Propón el nombre de la clase.
- Escribe al menos 4 atributos.
- Escribe al menos 3 métodos.
- Propón un constructor básico.
- Explica en 2 o 3 líneas qué detalles del objeto real dejaste fuera y por qué.

Clase propuesta	
Atributos	
Métodos	
Constructor	
Justificación	



Criterios de evaluación sugeridos

Categoría	% logro	Descripción de niveles de logro
Identificación de la clase	25%	Reconoce correctamente el objeto principal del caso y lo traduce a una clase coherente.
Selección de atributos	25%	Elige atributos relevantes, suficientes y relacionados con la responsabilidad de la clase.
Selección de métodos	25%	Propone métodos que representan acciones posibles y útiles para el problema.
Justificación y claridad	25%	Explica por qué selecciona ciertos elementos y redacta de forma ordenada y comprensible.

Resumen de avance de la guía

Bloque	Cantidad de ejercicios	Producto esperado	Observaciones
Casos 1 a 3	4	Clases simples del entorno cotidiano	
Casos 4 a 6	4	Clases académicas y de servicios	
Casos 7 a 8	3	Clases con foco en registro y estados	
Casos 9 a 10	2	Clases finales de integración	
Total	13	13 propuestas de clases con atributos y métodos	

Cierre

Al finalizar esta guía, el estudiante debería ser capaz de observar una situación cotidiana, identificar un objeto relevante y transformarlo en una clase simple de Java mediante un proceso básico de abstracción.

Creación de un proyecto JavaFX básico en IntelliJ IDEA con FXML

Sigla	Asignatura	Experiencia de Aprendizaje
POO-JFX	Programación Orientada a Objetos	Crear y ejecutar una interfaz JavaFX básica en IntelliJ IDEA utilizando FXML, HelloApplication.java y HelloController.java.
Tiempo	Modalidad de Trabajo	Indicadores de Logro
90 minutos	Trabajo individual guiado en laboratorio	Identifica la estructura del proyecto, vincula el archivo FXML con su controlador y ejecuta correctamente la primera ventana de la aplicación.

Título

Objetivo de la actividad

En esta actividad desarrollarás paso a paso un proyecto JavaFX en IntelliJ IDEA usando la estructura básica generada por el asistente del IDE. La práctica se concentra en tres archivos principales: hello-view.fxml, HelloApplication.java y HelloController.java, con el propósito de comprender cómo se carga una interfaz FXML, cómo se asocia un controlador y cómo se muestra correctamente la primera ventana de la aplicación.

Resultados de aprendizaje esperados

- Reconocer la estructura base de un proyecto JavaFX creado en IntelliJ IDEA.
- Editar de manera correcta HelloApplication.java, HelloController.java y hello-view.fxml sin aplicar un patrón MVC formal.
- Ejecutar la aplicación y comprobar visualmente que la primera ventana se despliega sin errores.

Recursos necesarios

- IntelliJ IDEA con el complemento JavaFX habilitado.
- JDK instalado y configurado en IntelliJ IDEA.
- Conexión a internet para descargar dependencias o validar la documentación oficial si fuese necesario.
- JavaFX Scene Builder opcional para visualizar el archivo FXML, aunque esta guía puede completarse editando el archivo directamente.



Consideraciones previas

Aunque IntelliJ IDEA puede generar archivos adicionales como module-info.java, en esta actividad el trabajo se enfoca en la comprensión y edición de hello-view.fxml, HelloApplication.java y HelloController.java. Si el asistente del IDE crea otros archivos, se recomienda no eliminarlos; simplemente mantén el foco en los tres elementos principales solicitados.

Desarrollo paso a paso

Paso 1. Abre IntelliJ IDEA y verifica que el complemento JavaFX esté habilitado. Para ello, ingresa a la configuración del IDE, revisa la sección de plugins instalados y confirma que JavaFX esté activo. Esta validación evita problemas al momento de crear o editar archivos FXML.

Paso 2. Desde la pantalla inicial del IDE selecciona New Project. En el panel de generadores elige JavaFX, asigna un nombre al proyecto, define una ubicación de guardado y selecciona el JDK disponible. Si tu instalación de IntelliJ no muestra JavaFX, revisa el plugin o instala una versión del JDK compatible.

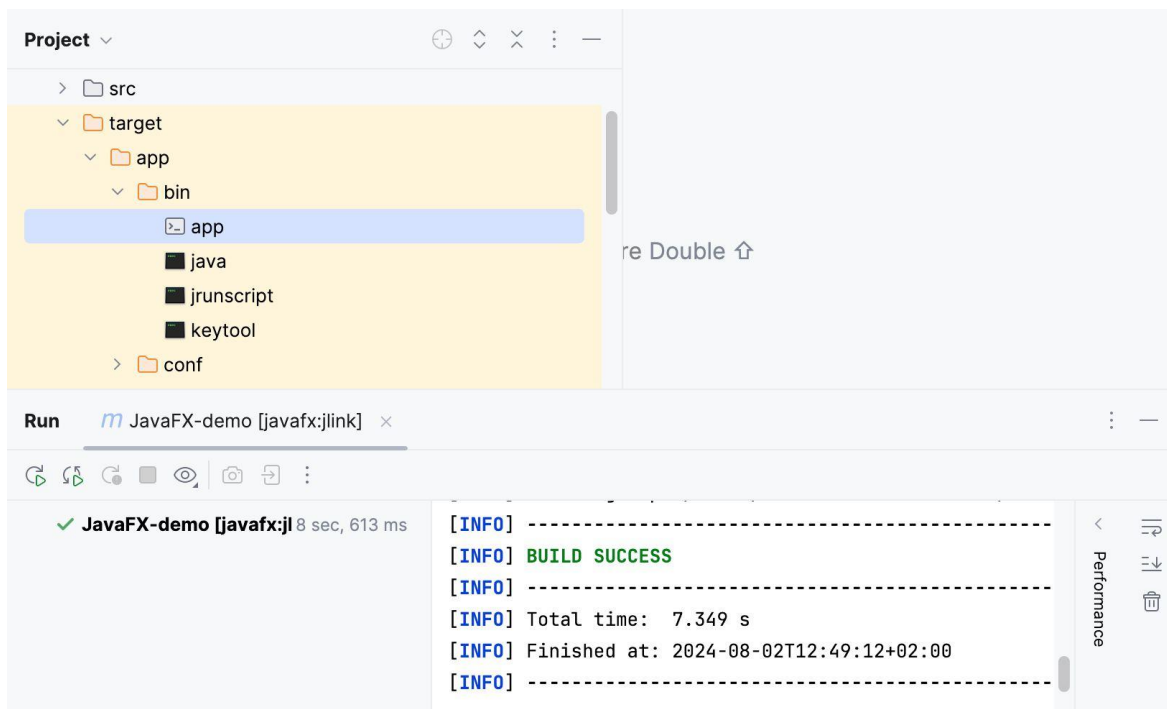


Figura 1. Evidencia visual del entorno de trabajo JavaFX en IntelliJ IDEA.

Fuente: captura real tipo IntelliJ proporcionada por JetBrains/Genspark.

Paso 3. Completa el asistente de creación del proyecto. IntelliJ generará una estructura base con un archivo FXML, una clase principal llamada HelloApplication y un controlador llamado HelloController. Esta estructura será suficiente para realizar la práctica sin incorporar una arquitectura MVC completa.

Paso 4. Ubica la clase HelloApplication.java. Su función es iniciar la aplicación, cargar el archivo hello-view.fxml mediante FXMLLoader, construir la escena y mostrarla en un Stage. Reemplaza el contenido por el código sugerido en esta guía para asegurar que la ventana tenga un título claro y un tamaño visible.

Paso 5. Abre `HelloController.java`. Este archivo actuará como el controlador básico de la vista. Aquí se declarará una etiqueta `Label` con la anotación `@FXML` y un método que responderá al evento del botón. El objetivo es comprobar que el controlador se enlaza correctamente con la vista `FXML`.

Paso 6. Edita `hello-view.fxml`. Define una estructura simple usando un `VBox` centrado, agrega un título, una etiqueta con `fx:id` y un botón con la propiedad `onAction`. El atributo `fx:controller` debe apuntar exactamente al paquete y nombre de la clase `HelloController`; si existe un error en esa ruta, la ventana no cargará correctamente.

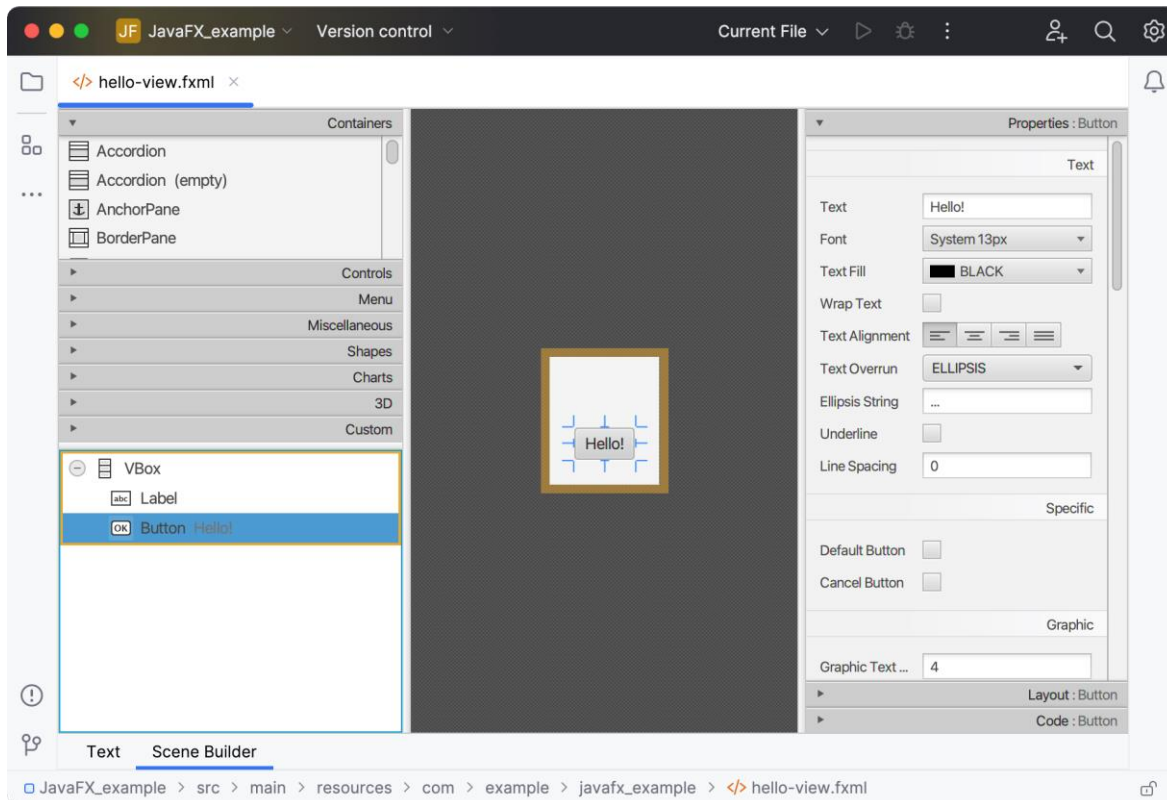


Figura 2. Visualización de `hello-view.fxml` mediante Scene Builder integrado en IntelliJ IDEA.

Fuente: captura real tipo IntelliJ proporcionada por JetBrains/Genspark.

Paso 7. Guarda todos los cambios y ejecuta la aplicación desde `HelloApplication.java` con la opción `Run`. Si la configuración es correcta, se abrirá una primera ventana titulada `Mi primera ventana JavaFX`. Al presionar el botón, el texto de la etiqueta deberá cambiar para confirmar la comunicación entre la vista y el controlador.

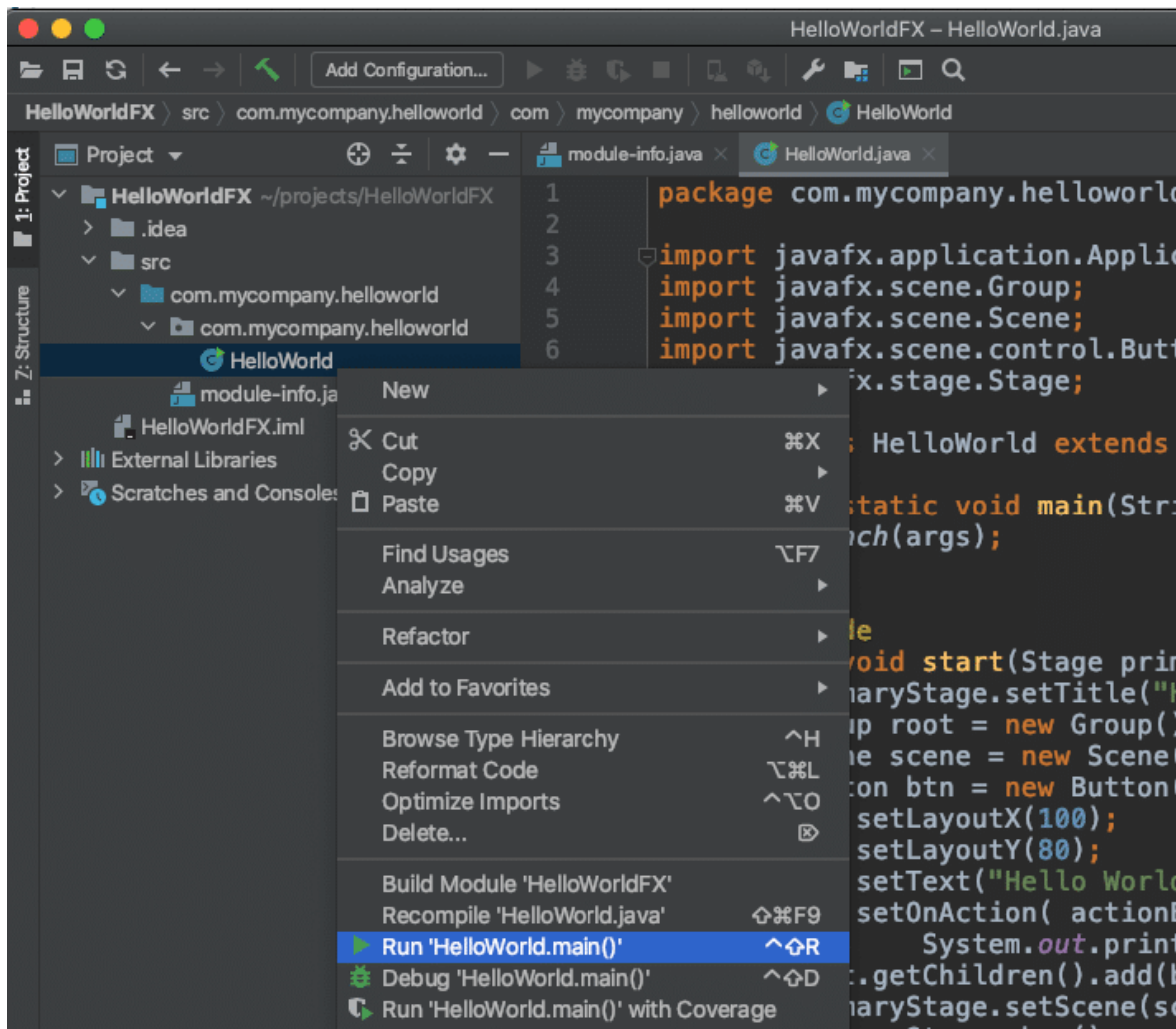


Figura 3. Ejecución rápida de la clase principal desde IntelliJ IDEA.

Fuente: captura real tipo IntelliJ proporcionada por JetBrains/Genspark.

Paso 8. Verifica el resultado final. La evidencia mínima esperada es la primera ventana visible, con el botón operativo y sin mensajes de error en la consola. Si el IDE genera una configuración temporal de ejecución, puedes reutilizarla para futuras pruebas sin volver a configurarla manualmente.

Código de referencia para la actividad

Archivo 1. HelloApplication.java

```
package com.example.javfxbasico;

import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Scene;
import javafx.stage.Stage;

import java.io.IOException;

public class HelloApplication extends Application {
    @Override
    public void start(Stage stage) throws IOException {
        FXMLLoader fxmlLoader = new FXMLLoader(
            HelloApplication.class.getResource("hello-view.fxml")
        );
        Scene scene = new Scene(fxmlLoader.load(), 600, 400);
        stage.setTitle("Mi primera ventana JavaFX");
        stage.setScene(scene);
        stage.show();
    }

    public static void main(String[] args) {
        launch();
    }
}
```

Archivo 2. HelloController.java

```
package com.example.javfxbasico;

import javafx.fxml.FXML;
import javafx.scene.control.Label;

public class HelloController {

    @FXML
    private Label welcomeText;

    @FXML
    protected void onHelloButtonClick() {
        welcomeText.setText("¡La primera ventana JavaFX se cargó correctamente!");
    }
}
```

Archivo 3. hello-view.fxml

```
<?xml version="1.0" encoding="UTF-8"?>
<?import javafx.geometry.Insets?>
<?import javafx.scene.control.Button?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.layout.VBox?>

<VBox alignment="CENTER" spacing="15.0"
      xmlns="http://javafx.com/javafx/22"
      xmlns:fx="http://javafx.com/fxml/1"
      fx:controller="com.example.javfxbasico.HelloController">
```

```
<padding>
    <Insets top="20" right="20" bottom="20" left="20" />
</padding>

<Label text="JavaFX en IntelliJ IDEA"
      style="-fx-font-size: 22px; -fx-font-weight: bold;" />

<Label fx:id="welcomeText"
      text="Presiona el botón para probar el controlador."
      wrapText="true" />

<Button text="Mostrar mensaje" onAction="#onHelloButtonClick" />
</VBox>
```

Recomendaciones de trabajo

- Mantén el mismo nombre de paquete en HelloApplication.java y HelloController.java para evitar inconsistencias en la referencia del controlador.
- Guarda los cambios antes de ejecutar, especialmente después de editar el archivo hello-view.fxml.
- Si utilizas Scene Builder, confirma que los cambios también se reflejen correctamente en la pestaña de texto del archivo FXML.
- Utiliza nombres descriptivos en títulos y mensajes para que la evidencia final sea clara al momento de revisar la actividad.

Errores comunes y forma de solucionarlos

- Error al cargar el FXML: normalmente ocurre porque la ruta de getResource("hello-view.fxml") es incorrecta o el archivo no está en la carpeta esperada de recursos.
- Error en fx:controller: sucede cuando el nombre del paquete o de la clase HelloController no coincide exactamente con el valor indicado en el archivo FXML.
- Error de inyección @FXML: aparece cuando el fx:id del Label no coincide con el nombre del atributo definido en HelloController.java.
- La ventana no se abre al ejecutar: revisa que estés corriendo HelloApplication.main() y que el JDK seleccionado sea el correcto.
- Problemas con JavaFX en proyectos modulares: si el IDE generó module-info.java y surge un error, verifica que existan los módulos javafx.controls y javafx.fxml, además de la apertura del paquete al módulo javafx.fxml.

Evidencias esperadas

- Captura del proyecto creado en IntelliJ IDEA con los archivos principales visibles.
- Código completo y funcional de HelloApplication.java, HelloController.java y hello-view.fxml.
- Captura de la primera ventana en ejecución y breve explicación del resultado obtenido.

Referencias sugeridas

Documentación oficial consultada: [JetBrains: Create a new JavaFX project](#)

Apoyo para edición FXML: [JetBrains: Configure JavaFX Scene Builder](#)

Marco conceptual sobre FXML: [OpenJFX: Introduction to FXML](#)

Categoría	% logro	Descripción niveles de logro
Configuración del proyecto	30%	Crea el proyecto JavaFX, selecciona el JDK correcto y reconoce la estructura inicial generada por IntelliJ IDEA.
Implementación de FXML y controlador	40%	Edita correctamente hello-view.fxml, HelloApplication.java y HelloController.java, manteniendo coherencia entre fx:id, onAction y fx:controller.
Ejecución y evidencia de la primera ventana	30%	Ejecuta la aplicación sin errores y presenta evidencia clara de la primera ventana funcionando en IntelliJ IDEA.

Lista de cotejo complementaria

Criterio	Logrado	Medianamente logrado	No logrado
Crea el proyecto JavaFX y organiza adecuadamente su estructura inicial.			
Edita correctamente HelloApplication.java, HelloController.java y hello-view.fxml.			
Configura sin errores fx:id, fx:controller y onAction.			
Ejecuta la aplicación y muestra la primera ventana funcional.			
Entrega evidencias ordenadas y explica al menos dos errores comunes con su solución.			
Total			100%

Observación final: esta guía propone una práctica inicial y funcional. El foco no está en la complejidad del diseño visual, sino en comprender el flujo mínimo entre aplicación, vista FXML y controlador para que la primera ventana de JavaFX se muestre correctamente.

Documento preparado para revisión académica y exportación. Mantiene la lógica visual del formato base, incorpora desarrollo guiado y utiliza referencias reales tipo IntelliJ IDEA para apoyar la comprensión del proceso.